Dossier de conception — API Bibliothèque

Projet fil rouge API REST — Bachelor CODA Orléans

Q1 — Ressources et URLs

1. Les trois ressources : les auteurs, les livres et les emprunts.

2. URLs :

Ressource	Collection	Élément d'id 3
Auteurs	`http://localhost:8080/api/v1/auteurs`	`http://localhost:8080/api/v1/auteurs/3`
Livres	`http://localhost:8080/api/v1/livres`	`http://localhost:8080/api/v1/livres/3`
Emprunts	`http://localhost:8080/api/v1/emprunts`	`http://localhost:8080/api/v1/emprunts/3`

3. Quatre règles de nommage :

- Pas de verbe dans l'URL (c'est la méthode HTTP qui dit l'action) — incorrect : `/api/v1/getLivres`
- Nom de ressource au pluriel — incorrect : `/api/v1/livre/3`
- Minuscules, mots séparés par des tirets — incorrect : `/api/v1/emprunts/EnRetard`
- L'identifiant d'un élément est dans le chemin, pas en paramètre — incorrect : `/api/v1/auteurs?id=3`

(On peut aussi citer le versionnement dans l'URL — incorrect : `/api/auteurs`.)

Q2 — Méthodes HTTP

1. On crée un nouvel emprunt dont l'id est choisi par le serveur : c'est le rôle de POST sur la collection. PUT sert à remplacer une ressource dont on connaît déjà l'URL ; `PUT /emprunts` voudrait dire « remplacer toute la collection », ce qui n'a pas de sens. En plus, chaque POST crée un nouvel emprunt (non idempotent), alors que PUT doit être idempotent.

2. PATCH sert à modifier **une partie** d'une ressource. Le retour change seulement `rendu` et `dateRetourEffective`, le réapprovisionnement change seulement `stock`. Avec PUT, le client devrait renvoyer l'objet complet pour le remplacer en entier.

3. Le chemin sert à identifier une ressource ; « harry » n'est pas une ressource, c'est un critère de filtre. Les paramètres de requête sont faits pour ça : ils sont optionnels, on peut les combiner (`?titre=harry&page=0&size=5`) et ils gèrent bien les textes avec espaces.

Q3 — Idempotence

1. Premier envoi : **201**, deuxième envoi : **201**. Deux emprunts sont créés (id 4 et 5). Le stock du livre 1 passe de 5 à **3**.

2. Premier envoi : **204** (livre supprimé), deuxième envoi : **404** (il n'existe plus). État final : le livre 6 n'est plus en base.

3. DELETE est idempotent : après un ou deux appels, la base est dans le même état (le livre 6 est supprimé), seul le code change ; alors que chaque POST crée un emprunt de plus et baisse encore le stock.

Q4 — Codes de statut

1.

#	Situation	Code
a	POST /auteurs avec un corps valide	201
b	DELETE /livres/2 (le livre existe)	204
c	GET /livres/999	404
d	POST /livres avec un corps valide mais “auteurId”: 999	400
e	POST /livres avec l'ISBN du livre 1	409
f	POST /emprunts pour le livre 8 (« Le Hobbit »)	409
g	POST /auteurs avec “nom”: “”	400
h	DELETE /auteurs/1 (Victor Hugo)	409

2. En **c**, c'est la ressource désignée par l'URL (`/livres/999`) qui n'existe pas : 404. En **d**, l'URL `/livres` existe bien ; c'est une donnée du corps (`auteurId`) qui est incorrecte : 400.

3. Le code HTTP est la première chose que lit le client (et les outils : navigateur, Postman, caches, monitoring). Avec 200, tout le monde croit que la requête a réussi ; le client devrait lire

et interpréter le corps pour découvrir l'erreur, et chaque API inventerait son propre format. On casse le sens standard de HTTP.

Q5 — Représentations JSON

1. `GET /livres/1` → 200

```
{
    "id": 1,
    "titre": "Les Misérables",
    "isbn": "978-2-07-040850-4",
    "prix": 12.5,
    "anneePublication": 1862,
    "stock": 5,
    "auteurId": 1
}
```

2. Code **201 Created**

```
{
    "id": 4,
    "livreId": 2,
    "emprunteur": "Alice Durand",
    "dateEmprunt": "2026-11-02",
    "dateRetourPrevue": "2026-11-16",
    "dateRetourEffective": null,
    "rendu": false,
    "enRetard": false
}
```

Le stock du livre 2 passe de 3 à **2**.

Q6 — Pagination

1. `http://localhost:8080/api/v1/livres?page=1&size=3&sort=prix,desc` (la 2e page a le numéro 1, car on compte à partir de 0).

2. `page.totalElements` = **8** et `page.totalPages` = **3** (3 + 3 + 2).

3. Deux raisons :

- Performance : le serveur et la base ne chargent pas des milliers de lignes d'un coup.
- Réseau et affichage : des réponses plus légères et plus rapides, surtout sur mobile, et le client n'affiche de toute façon qu'une page à la fois.

Q7 — Sans état

1. Le serveur ne garde aucune information sur le client entre deux requêtes (pas de session). Chaque requête doit contenir tout ce qu'il faut pour être traitée seule.

2. Comme le serveur ne « se souvient » pas de qui est l'utilisateur, il ne peut pas deviner qui emprunte. La requête doit donc porter elle-même le nom et l'email de l'emprunteur.

Q8 — DTOs

1. Ces valeurs ne doivent pas venir du client : l'id est généré par la base, et les dates et `rendu` sont calculés par le serveur (RM5). Si le client pouvait les envoyer, il pourrait tricher (par exemple choisir une date de retour dans un an, ou créer un emprunt déjà rendu).

2. L'email est une donnée personnelle (RM8, RGPD). Il est utile à la bibliothèque, mais il n'a rien à faire dans les réponses que peut lire n'importe quel client.

3. Deux problèmes :

- On exposerait l'email de l'emprunteur : fuite de données personnelles.
- Le contrat de l'API serait collé à la structure de la base : renommer ou ajouter une colonne casserait les clients. Et on ne pourrait pas ajouter de champ calculé comme `enRetard`.

Q9 — Tester avec curl

1.

```
curl -i http://localhost:8080/api/v1/auteurs/1
```

`-i` affiche la ligne de statut et les en-têtes avant le corps.

2. L'en-tête `Content-Type`, avec la valeur `application/json`.

3.

```
curl -i -X POST http://localhost:8080/api/v1/auteurs \
     -H "Content-Type: application/json" \
     -d '{"nom": "Verne", "prenom": "Jules", "nationalite":
"Française"}'
```

- `-i` : affiche le code de statut et les en-têtes de la réponse (on doit voir 201) ;
- `-X POST` : choisit la méthode HTTP POST ;
- `-H "Content-Type: application/json"` : dit au serveur que le corps envoyé est du JSON ;
- `-d '...'` : le corps de la requête (le JSON de l'auteur).

Sous Windows (cmd), on écrit tout sur une ligne et on échappe les guillemets du JSON : `curl -i -X POST http://localhost:8080/api/v1/auteurs -H "Content-Type: application/json" -d "{\"nom\": \"Verne\", \"prenom\": \"Jules\", \"nationalite\": \"Française\"}"`